Cross-Origin-Resource-Sharing (CORS)

Darf man per HTTP frei auf Ressourcen im Internet frei zugreifen?

1. Browser lädt eine Seite von www.example.com
2. Seite will Service auf www.example.com aufrufen.
3. Das ist erlaubt.



Same-Origin Policy: Der Zugriff auf Services des Hosts, von dem die Webseite geladen wurde, ist erlaubt.

1. Nutzer öffnet www.shop.com und logged sich ein. Login wird in Cookie gespeichert.
2. Nutzer öffnet eine Phishing-Seite. Diese ruft nun mit dem Cookie des Nutzers des Services des Shops auf und kauft dort ohne Wissen des Nutzers ein.



Diesen Zugriff auf Ressourcen einer anderen Web-Site nennt man **Cross-Origin-Request** und wird von allen Browsern unterbunden!

Was tun?

Der Server muss Cross-Origin Requests erlauben:

```
app.use(cors({
    origin: 'http://localhost:63343',
    methods: ['GET', 'POST', 'PUT', 'DELETE'],
    allowedHeaders: ['Content-Type', 'Authorization']
}));
```

'*', falls man alle Requests zulassen will. Vorsicht!

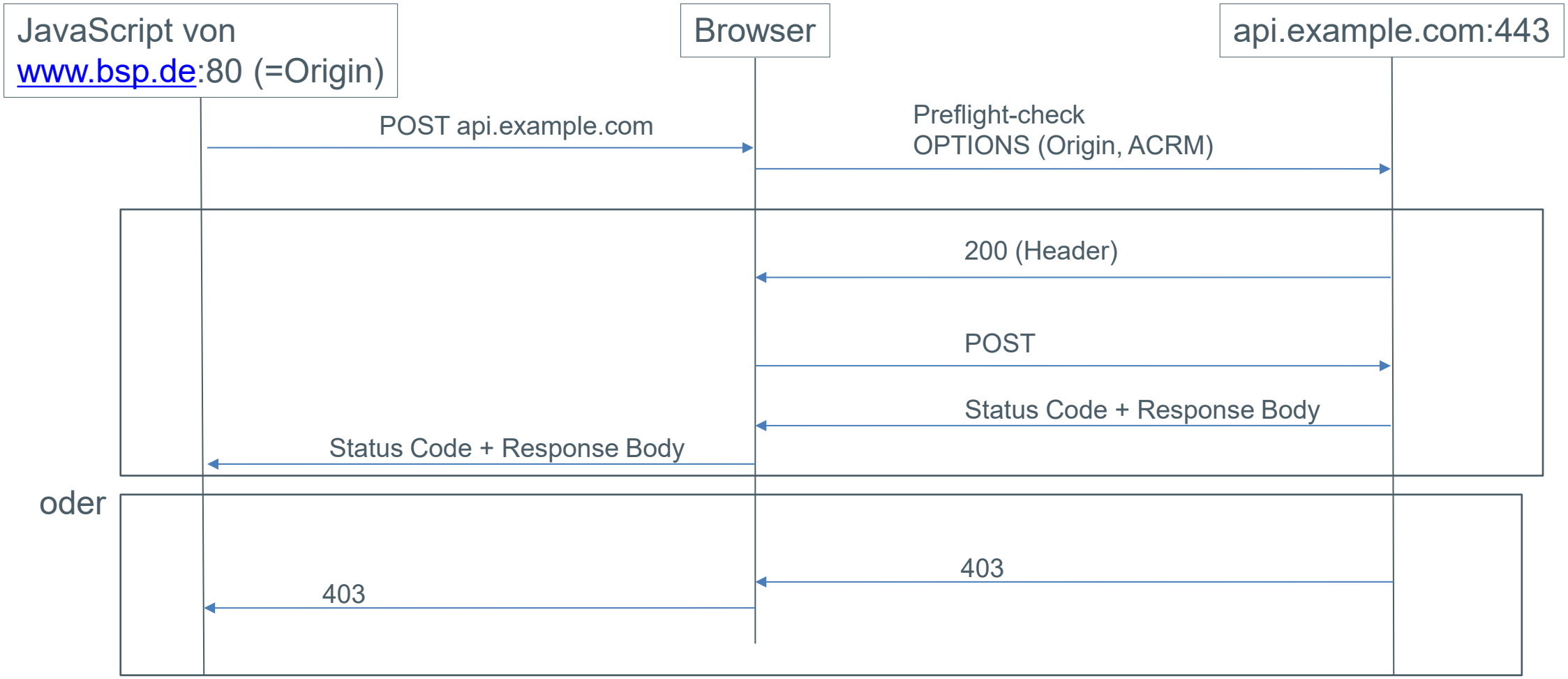
Zulässige Verben

Vorher müssen Sie in package.json eintragen:

Und `npm install` ausführen.

```
"dependencies": {
  "express": "^4.18.2",
  "cors": "^2.8.5"
},
"devDependencies": {
  "@types/express": "^4.17.17",
  "@types/node": "^20.6.3",
  "@types/cors": "^2.8.13",
  "typescript": "^5.2.2"
}
```

Ablauf


JavaScript von
www.bsp.de:80 (=Origin)
Browser
api.example.com:443
POST api.example.com
Preflight-check
OPTIONS (Origin, ACRM)
200 (Header)
POST
Status Code + Response Body
Status Code + Response Body
oder
403
403

Ablauf

- Wenn `mode:CORS` gesetzt ist, führt der Browser einen **Preflight-Check** durch. Selbst wenn dies nicht gesetzt ist, wird der Browser bei „unsafe“ Headern oder Verben außer `GET`, `POST`, `HEAD` und cross-origin Requests einen **Preflight-Check** durchführen.
- Mit dem HTTP-Verb `OPTIONS` werden dann gesendet:
  - `Origin` (Host und Port, von dem der Browser das Skript erhalten hat (im Beispiel: www.bsp.de:80)
  - `Access-Control-Request-Method`: Verb, das das Skript nutzen will (im Beispiel: POST)
  - `Access-Control-Request-Headers` : Weitere Header Felder

- Server überprüft, ob der Origin zu seinem `Access-Control-Allow-Origin` passt und ob die Methode erlaubt ist.
- Und sendet im Erfolgsfall eine Response mit Status Code `204` und `Access-Control-Allow-Origin` : www.bsp.de:80 (oder * oder …), sonst `403 (Forbidden)`

- Mehr Infos bei: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers

Code Snippets

Client (from localhost:63342)

```
const res = await fetch('http://localhost:3000/add', {
    method: 'put',
    mode: 'cors', // This sets CORS mode, not required here
    headers: {
        "Content-type": "application/json"
    },
    body: JSON.stringify({a, b, c})
});
```

Server (on localhost:3000)

```
import express, { Express, Request, Response } from 'express';

import cors from 'cors';

app.use(express.json());


app.use(cors({
        origin: 'http://localhost:63343',
        methods: ['GET', 'POST',  'PUT', 'DELETE'],
        allowedHeaders: ['Content-Type', 'Authorization']
    }));

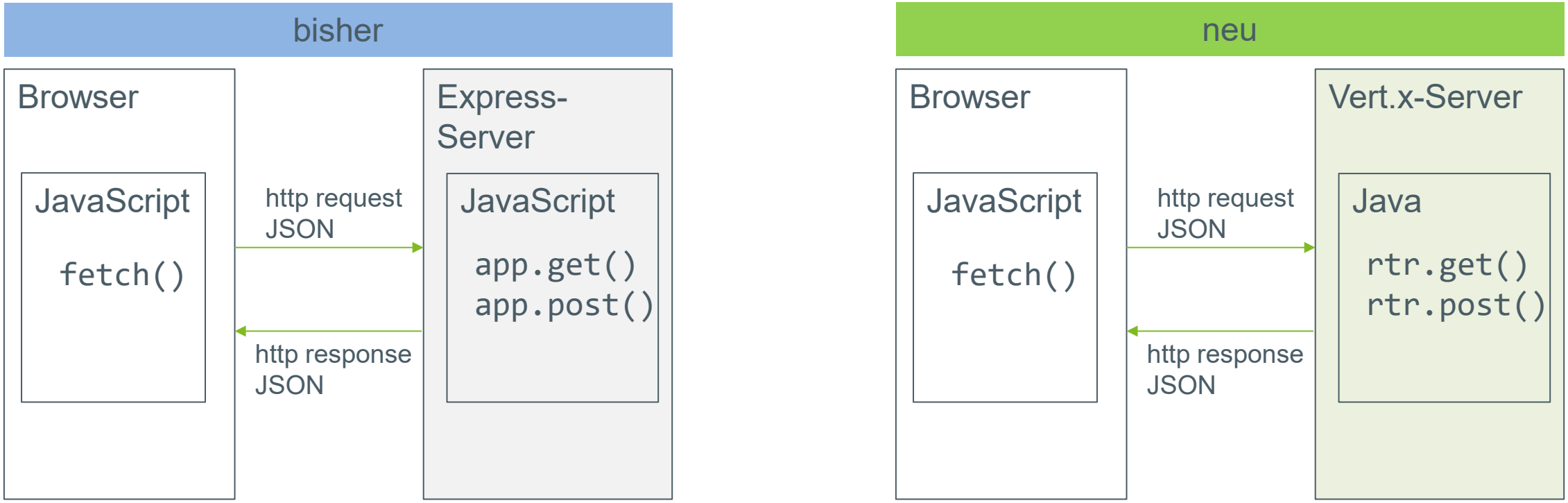


app.listen(...);

app.post("/add, …);
```

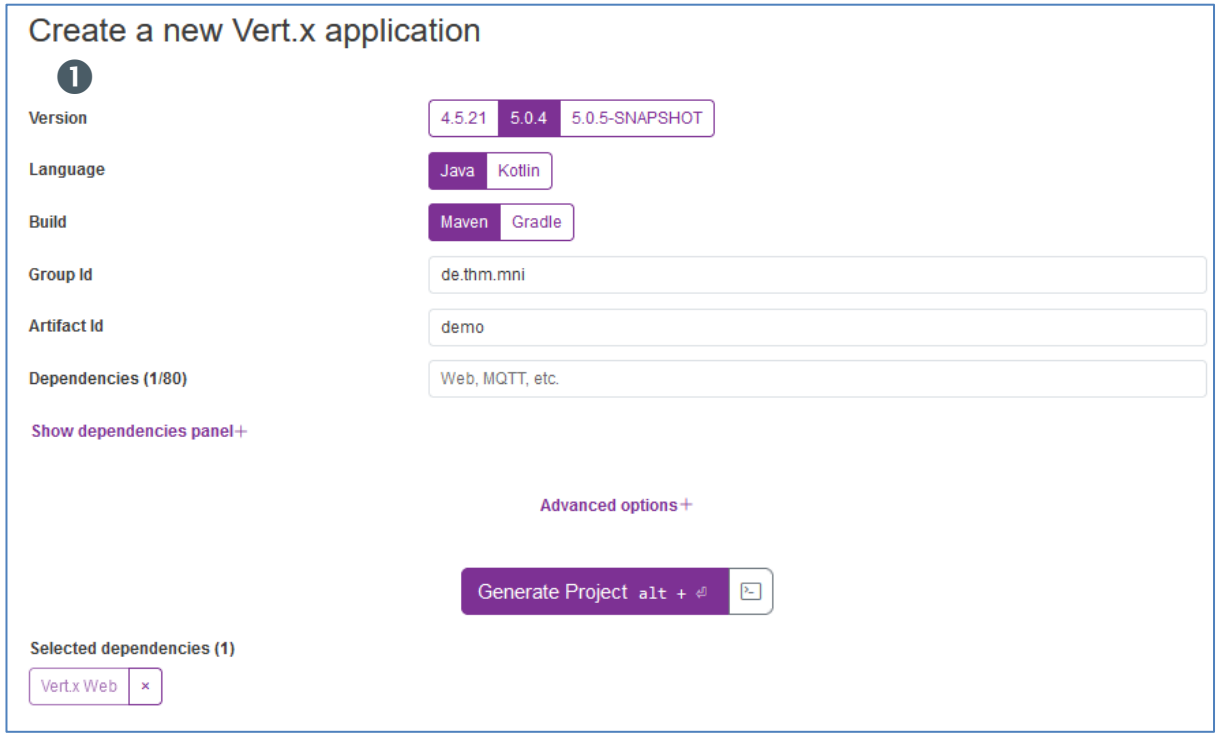
Jetzt mit Java …

- Da HTTP ein offenes Protokoll ist, können Services in jeder Programmiersprache implementiert werden.
- Wir werden daher jetzt das gleiche Beispiel nochmals mit Java umsetzen.



Eclipse Vert.x

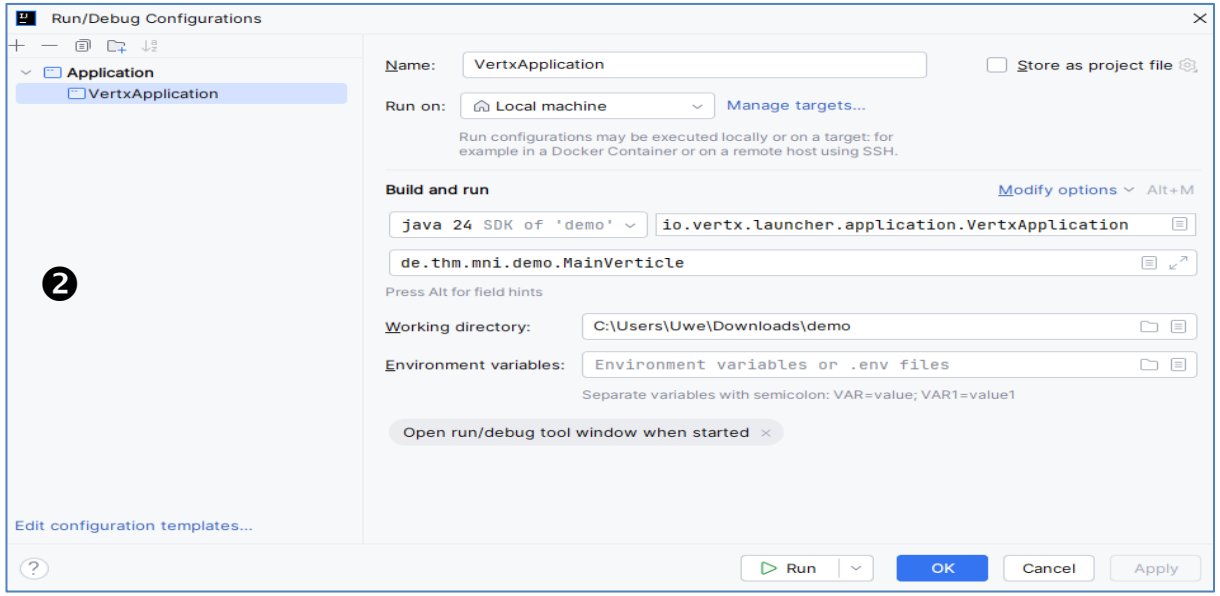
- https://vertx.io/get-started/ durchlesen und den Generator starten
- zip-file in neues Verzeichnis entpacken
- IntelliJ -> File -> Open -> <Verzeichnis>/pom.xml



- IntelliJ -> Run ->Edit Configurations -> + -> Application Konfiguration eintragen
- Starten (grüner Pfeil oben rechts)

=> `HTTP server started on port 8888 …`

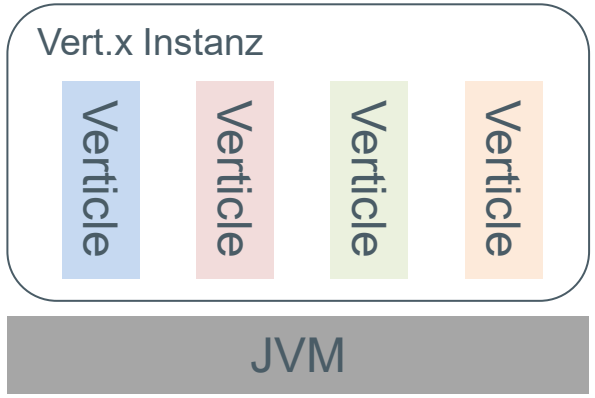
- http://localhost:8888 aufrufen



Vert.x Übersicht

- Vert.x ist eine leichtgewichtige asynchrone Plattform, die in vielen Teilen ähnlich zu Express ist

- **Verticles** sind das wichtigste Konzept
- Starter erzeugt ein Verticle („MainVerticle“)
- Ein Verticle ist ein Code-Paket, welches von der Vert.x Instanz ausgeführt wird
- Verticles kümmern sich um die Bearbeitung von Ereignissen wie z.B. eingehenden HTTP-Requests



- **Router** leben in einem Verticle und definieren die HTTP-Routen (siehe nächste Seiten)
- Typischerweise erzeugt ein Verticle einen Router

- **Handler** implementieren die Logik für die Verarbeitung der Routen
- Ein Router verwaltet i.A. mehrere Handler für dieselbe Route.
- Ein Handler kann die Verarbeitung beenden (`end()`) oder an den nächsten Handler weitergeben (`next()`)

Statische Inhalte

```
Router router = Router.router(vertx);

router.route("/static/*").handler(
  StaticHandler.create("public")
);
```

Beispiel: **GET /static/bla.txt**
Gibt automatisch den Inhalt der Datei bla.txt aus dem Verzeichnis `public` zurück

Statische Inhalte liegen in einem Verzeichnis `static`.
Dieses Verzeichnis liegt (bei Maven und Gradle) unter `src/main/recources`.

```
GET http://localhost:8888/static/bla.txt
bla bla
```

Dynamische Inhalte mit statischen Routen

```
Router router = Router.router(vertx);

router.get("/datum").handler(rc -> {

  Date datum = new Date();

  rc.response()
  .setStatusCode(200)
  .putHeader("content-type", "text/plain")
  .end("Heutiges Datum: " + datum);

});
```

Beispiel: **GET /datum**
Gibt das heutige Datum zurück

```
GET http://localhost:8888/datum
Heutiges Datum: Tue Sep 09 14:12:32 CEST 2025
```

GET mit Query-Parametern

Beispiel:.

Beispiel: **GET /echo?message=Hallo**
soll „Echo: Hallo“ zurückliefern

```
Router router = Router.router(vertx);

router.get("/echo").handler(rc -> {
  String message = rc.request().getParam("message");
  rc.response().setStatusCode(200)
    .putHeader("content-type", "text/plain")
    .end("Echo: " + message);
});
```

Parameter message mit getParam() abfragen

```
GET http://localhost:8888/echo?message=Hallo
Echo: Hallo
```

GET mit Path-Parametern

Beispiel:.

```
Router router = Router.router(vertx);

router.get("/ping/:message").handler(rc -> {
  String message = rc.pathParam("message");
  rc.response().setStatusCode(200).
    .putHeader("content-type", "text/plain")
    .end("Pong: ", + message);
});
```

```
GET http://localhost:8888/ping/Hallo
Pong: Hallo
```

Beispiel: **GET /ping/Hallo**
soll “Pong: Hallo” zurückgeben.

] Parameter message mit pathParam() abfragen

Route mit Request-Body

POST http://localhost:8888/double
Content-Type: application/json

{"eingabe": 24}



{"ausgabe": 48}

```
Router router = Router.router(vertx);

router.post("/double").handler(rc -> {
   try {
         JsonObject jsonBody = rc.body().asJsonObject();
         int eingabe = jsonBody.getInteger("eingabe");
         JsonObject ergebnis = new JsonObject().put("ausgabe", eingabe * 2);
         rc.response().setStatusCode(200)
           .putHeader("content-type", "application/json")
           .end(ergebnis.encode());
       }
       catch (Exception e) {
         rc.response().setStatusCode(400).end("Ungültiges JSON");
       }
```

Adder-Beispiel mit Vert.x

```
public Future<?> start() {
    int port = 8888;
    Router router = Router.router(vertx);

    router.post("/add")
      .handler(BodyHandler.create()).handler(this::add);

    return vertx
      .createHttpServer()
      .requestHandler(router)
      .listen(port).onSuccess(http -> {
        System.out.println("HTTP server started on port 8888");
    });
  }

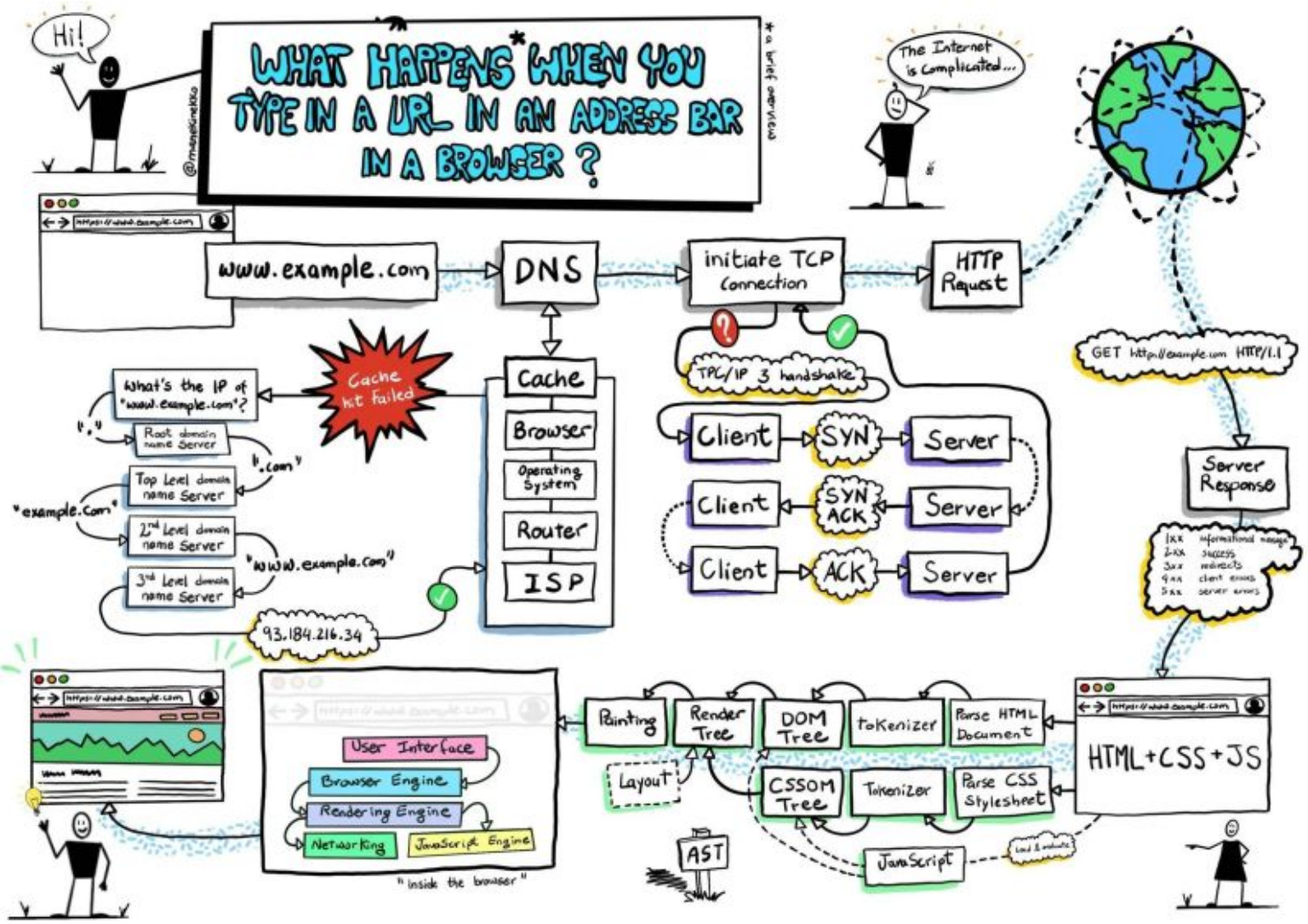
  private void add(RoutingContext rc) {
      try {
        JsonObject jsonBody = rc.body().asJsonObject();
        int a = jsonBody.getInteger("a");
        int b = jsonBody.getInteger("b");
        int c = jsonBody.getInteger("c");
        JsonObject ergebnis = new JsonObject().put("result", a + b + c);
        rc.response().setStatusCode(200)
          .putHeader("content-type", "application/json")
          .end(ergebnis.encode());
      } catch (Exception e) { rc.response().setStatusCode(400).end("Ungültiges JSON");  }
    }
}
```

Methodenreferenz (`::`) : Vert.x soll, wenn die Route `/add` mittels POST aufgerufen wird, auf diesem Objekt die Methode `add()` aufrufen.

POST http://localhost:8888/add
Content-Type: application/json

{ "a" : 3, "b" : 4, "c" : 5 }

{ "result" : 12 }


Hi!
WHAT HAPPENS WHEN YOU TYPE IN A URL IN AN ADDRESS BAR IN A BROWSER ?
The Internet is complicated...
www.example.com
DNS
initiate TCP connection
HTTP Request
GET http://example.com HTTP/1.1
TPC/IP 3 handshake
Client
SYN
Server
SYN ACK
ACK
Cache
Browser
Operating System
Router
ISP
Cache hit failed
What's the IP of "www.example.com"?
Root domain name Server
Top Level domain name Server
2nd Level domain name Server
3rd Level domain name Server
"example.com"
".com"
"www.example.com"
93.184.216.34
Server Response
HTML+CSS+JS
Parse HTML Document
DOM Tree
Render Tree
Painting
Layout
Parse CSS Stylesheet
CSSOM Tree
JavaScript
AST
User Interface
Browser Engine
Rendering Engine
Networking
JavaScript Engine
"inside the browser"

Fertig?